Assignment A5 : Using the Inference Service

Individual assignment • 20% individual-report component Released: 17 Dec 2026 Due: 9 Jan 2027, 23:59 (after the break)

Objective. Write a batched client for the course inference service, profile how latency scales with prompt and generation length, and produce a defensible token/compute budget for your group project. Inference has a cost, and you can measure it.

From the lecture

The lab for your last lecture, **LLM Serving** (14 Jan, Anand): prefill vs decode, the KV cache, and what they cost. Here you measure it directly and turn it into a project budget. **Pre-watch:** video #13 LLM Serving on the GPU, #08 Decoding.

At a glance — input, output, how it's evaluated

Input the course inference service (endpoint + credentials on Brightspace) and a set of fixed prompts.

Output `a5_latency.csv` (`prompt_tokens, gen_tokens, decoding, temperature, latency_s`) and `a5_budget.json`.

Evaluation the CSV is checked for schema, grid coverage, and the expected trend (latency rising with generation length); the budget arithmetic must be consistent with your own measurements. *Metrics:* latency (wall-clock), per-token decode-vs-prefill cost, and the token + wall-clock budget.

Data — a public Kaggle competition. All assignments build on **Jigsaw Unintended Bias in Toxicity Classification** (`jigsaw-unintended-bias-in-toxicity-classification`). Join the competition on Kaggle, then fetch it with the Kaggle API or via `starter/load_data.py` (which uses `kagglehub`). From its `train.csv` we derive, through the starter helpers:

- **Classification** — binary toxicity (`target` ≥ 0.5 = toxic) from `comment_text`, with the **identity columns** (`male`, `female`, `black`, `white`, `christian`, `muslim`, ...) as the **subgroups** for fairness analysis. Helper: `load_textclf()` → train / validation / test splits.
- **Raw text for LM work** — a fixed sample of `comment_text` with a held-out split. Helper: `load_lm()` → (train, held-out) lines.

Models come from HuggingFace, as usual. `check_submission.py` validates your output files before you submit; the grading labels for the `test` split are held out by the teaching team. *Content note: the comments can contain offensive language.* **Service.** Credentials and the base URL for the course inference endpoint are on Brightspace (`service/README`). It exposes an OpenAI-style `/generate` API; a thin client stub is in `starter/`.

Scope

Required core: the batched client, the prompt×generation latency sweep, and the project budget estimate (tasks 1, 2, 4) — the measure-the-cost core.

Optional: the greedy-vs-sampled decoding comparison (task 3). Insightful, but the cost lesson and the budget stand on the sweep alone.

1. Tasks

1. **Batched client.** Write a client that sends requests in batches and handles retries/timeouts.

2. **Latency sweep.** Vary **prompt length** × **generation length** over a fixed grid and record latency per cell (median over repeats). Plot the surface (two line families).
3. [optional] **Decoding comparison.** On a fixed set of prompts, compare **greedy** vs **sampled** output at **two temperatures**; report a simple quality/diversity observation alongside latency.
4. **Budget.** Estimate the **token and wall-clock budget** your group project will need, from your measured per-token costs, in the provided template.

Runs on your laptop

This is a **client-only** assignment: the model runs on the service, your laptop just sends HTTP requests and plots results (`requests`/`httpx` + `matplotlib`/`pandas`). No local model, no GPU. Be considerate: keep total request volume within the per-student quota stated on Brightspace.

Deliverable & auto-grading

Submit two files:

- `a5_latency.csv` — columns `prompt_tokens`, `gen_tokens`, `decoding`, `temperature`, `latency_s` (the greedy sweep is required; sampled rows only if you do the optional comparison).
- `a5_budget.json` — filled from the fixed template: `{ tokens_in, tokens_out, est_total_tokens, est_wallclock_min, assumptions }`.

Grading: `a5_latency.csv` is checked for the required schema, grid coverage, and internally consistent trends (latency rising with generation length); `a5_budget.json` must be complete and its arithmetic must check out against your own CSV.

What is graded. The *shape* of your results — latency rising with generation length, roughly flat in prompt length, and a sensible decode/prefill per-token ratio — not the absolute milliseconds (those depend on the server). Report end-to-end wall-clock latency, and state your concurrency/batch assumption in the budget (a sequential batch-1 estimate is an upper bound).

2. Findings prompt

Which costs more, reading or writing — and why? Use your prompt-length vs generation-length surface to justify the answer (prefill vs decode).

3. Reflection (required, max 200 words)

Submit `reflection.md` (or `.pdf`). It must answer these three fixed questions and include the AI-use declaration. The reflection is graded for honesty and insight, not length.

1. **What did you know before?** — your starting point on this topic.
2. **What was hardest, and how did you get unstuck?**
3. **What does your result say, and what would you try next?**

AI-use declaration. State which parts (if any) were generated with AI tools, and *how you verified them.* Using AI is allowed; undisclosed or unverified use is not.

4. Submission

Upload a single `zip` to Brightspace named `A5_<studentnumber>.zip`, containing the required output file(s) listed above, your `code/` folder, and `reflection.md`. Run the format checker in `starter/` on your zip first — it validates file names, schemas, and value ranges locally before you submit. Auto-grading scores your output file against the hidden reference; the format checker passing is a precondition for grading.